SVEUČILIŠTE U ZAGREBU
FAKULTET ORGANIZACIJE I INFORMATIKE
VARAŽDIN

Laura Krišković

APLIKACIJA ZA UPRAVLJANJE KORISNIČKIM RAČUNIMA

PROJEKT

TEORIJA BAZA PODATAKA

Varaždin, 2025.

SVEUČILIŠTE U ZAGREBU

FAKULTET ORGANIZACIJE I INFORMATIKE

V A R A Ž D I N

Laura Krišković

Matični broj: 0016155347

Studij: Baze podataka i baze znanja

APLIKACIJA ZA UPRAVLJANJE KORISNIČKIM RAČUNIMA

PROJEKT

Mentor:

doc. dr. sc. Bogdan Okreša Đurić

Varaždin, siječanj 2025.

Laura Krišković

Izjava o izvornosti

Izjavljujem da je ovaj projekt izvorni rezultat mojeg rada te da se u izradi istoga nisam koristio drugim izvorima osim onima koji su u njemu navedeni. Za izradu rada su korištene etički prikladne i prihvatljive metode i tehnike rada.

Autorica potvrdila prihvaćanjem odredbi u sustavu FOI Radovi

Sažetak

Ključne riječi:

Sadržaj

1. Opis aplikacijske domene 1

2. Teorijski uvod 3

2.1. Različiti pristupi bazama podataka 3

2.2. Objektno-relacijska baza podataka 3

2.2.1. Usporedba s drugim pristupima 4

3. Model baze podataka 5

4. Implementacija 7

5. Primjeri korištenja 14

6. Zaključak 19

Popis literature 19

Popis slika 21

7. Prilog 22

1. Opis aplikacijske domene

Aplikacija koju sam napravila osmišljena je kao sustav za upravljanje školskim zapisima, s posebnim naglaskom na upravljanje ocjenama i interakciju između različitih korisničkih uloga. Sustav omogućava sigurno i učinkovito evidentiranje, pregled i administraciju podataka o učenicima, ocjenama i korisnicima sustava. Ključni cilj aplikacije je osigurati transparentnu komunikaciju između učenika, roditelja i učitelja, uz administrativnu podršku za održavanje sustava.

Koncepti i relacije

Korisnici:

- **Učitelji:** Imaju mogućnost unositi ocjene učenicima na predmetima koje predaju. Također mogu dodavati bilješke uz ocjene za dodatno objašnjenje.
- **Učenici:** Mogu pregledavati vlastite ocjene i bilješke koje su vezane uz njih.
- **Roditelji:** Imaju uvid u ocjene i bilješke svoje djece, čime se osigurava transparentnost i omogućuje bolja komunikacija između obitelji i škole.
- **Administratori:** Upravljaju korisnicima sustava. Mogu dodavati nove korisnike, brisati postojeće te pregledavati metapodatke korisničkih računa (kao što su datumi kreiranja računa ili posljednje prijave).

Relacije:

- Svaki korisnik ima specifičnu ulogu u sustavu (učenik, roditelj, učitelj, admin).
- Svaki učitelj povezuje se s predmetima koje predaje i učenicima kojima upisuje ocjene.
- Roditelj je povezan s jednim ili više djece u sustavu.
- Administrator ima pristup svim korisničkim zapisima i njihovo upravljanje.

Specifičnosti domene:

- Svaki zapis o ocjeni uključuje datum, predmet, ocjenu i opcionalnu bilješku.
- Prava pristupa definirana su prema korisničkoj ulozi, čime se osigurava sigurnost i privatnost podataka.
- Sustav podržava pregled aktivnosti korisnika putem metapodataka (npr. praćenje posljednje prijave).

Motivacija za korištenje odabrane tehnologije

Za implementaciju ovog sustava odabrane su PostgreSQL baza podataka i Flask web okvir zbog njihove izvrsne kombinacije fleksibilnosti, skalabilnosti i jednostavnosti korištenja.

PostgreSQL je bio idealan izbor zbog svog objektno-relacijskog modela koji je omogućio precizno modeliranje kompleksnih odnosa između različitih tipova korisnika i njihovih aktivnosti. Nasljeđivanje mi je omogućilo da definiram različite vrste korisnika (učitelj, učenik, roditelj) kao podtipove osnovne klase "korisnik". Osim toga, PostgreSQL nudi napredne značajke poput JSONB tipova podataka, što mi je omogućilo da lako pohranjujem dodatne informacije o korisnicima, te podršku za složene upite koji su potrebni za analizu podataka. U usporedbi s drugim bazama podataka, kao što je MySQL, PostgreSQL nudi veću fleksibilnost i skalabilnost, što je ključno za budući razvoj aplikacije.

Flask je odabran kao web okvir zbog svoje jednostavnosti i ekstenzibilnosti. SQLAlchemy, popularni ORM alat, omogućuje mi da na jednostavan način manipuliram podacima u PostgreSQL bazi podataka. Flask-ova modularna arhitektura mi je omogućila da brzo postavim temelje aplikacije i postupno dodajem nove funkcionalnosti. U usporedbi s drugim okvirima, kao što je Django, Flask nudi veću slobodu u odabiru alata i biblioteka, što mi je omogućilo da prilagodim aplikaciju specifičnim potrebama ovog projekta. Za korištenje Flask frameworka unutar Python projekta, preporučuje se kreiranje virtualnog okruženja, što omogućava izolaciju ovisnosti specifičnih za projekt. Kreiranje virtualnog okruženja ostvaruje se pomoću naredbe `python -m venv naziv_okruženja`, gdje `naziv_okruženja` predstavlja naziv samog okruženja. Za aktivacija virtualnog okruženja na Linux sustavu morala sam koristiti naredbu `source naziv_okruženja/bin/activate`. Nakon aktivacije, unutar virtualnog okruženja moguće je instalirati Flask pomoću naredbe `pip install flask`, čime se osigurava da su svi paketi instalirani unutar tog okruženja. Nakon završetka rada, virtualno okruženje se deaktivira naredbom `deactivate`. **knjigaFlask**

Ova kombinacija tehnologija ne samo da zadovoljava trenutne zahtjeve aplikacije, već otvara i mogućnosti za buduća proširenja. Na primjer, u budućnosti bih mogla integrirati aplikaciju s nacionalnim obrazovnim platformama koristeći RESTful API-je. Također, mogla bih implementirati napredne analitike podataka kako bi se dobile uvide u učinkovitost nastavnog procesa.

U sljedećim poglavljima detaljnije ću opisati arhitekturu aplikacije, dizajn baze podataka, implementaciju ključnih funkcionalnosti i provedena testiranja.

2. Teorijski uvod

U ovom poglavlju će biti prikazani različiti pristupi bazama podataka te će biti obrazloženo zašto je za razvoj aplikacije za upis ocijena odabrana objektno-relacijska baza podataka PostgreSQL.

2.1. Različiti pristupi bazama podataka

Postoji nekoliko različitih pristupa dizajniranju i implementaciji baza podataka, svaki sa svojim specifičnim karakteristikama i područjima primjene. Neki od najčešćih pristupa uključuju: **skripta**

Relacijske baze podataka: Temelje se na relacijskom modelu podataka, koji podatke predstavlja u obliku tablica povezanih međusobno preko ključeva. Primjeri relacijskih baza podataka uključuju MySQL, PostgreSQL i Oracle.

Objektno-relacijske baze podataka: Kombiniraju najbolje karakteristike relacijskih i objektno-orijentiranih baza podataka, omogućujući pohranu i manipuliranje podacima na način koji je bliži objektno-orijentiranom programiranju. PostgreSQL je primjer objektno-relacijske baze podataka.

NoSQL baze podataka: Dizajnirane za velike količine neuređenih podataka, NoSQL baze podataka nude veliku skalabilnost i fleksibilnost. Primjeri uključuju MongoDB, Cassandra i CouchDB.

Hijerarhijske baze podataka: Organiziraju podatke u hijerarhijskoj strukturi, slično kao u datotečnom sustavu.

Mrežne baze podataka: Omogućuju složenije veze između podataka nego što je to slučaj u hijerarhijskim bazama.

Objektno-orijentirane baze podataka: Potpuno su usmjerene na objektno-orijentirani pristup, ali su manje popularne od objektno-relacijskih baza.

2.2. Objektno-relacijska baza podataka

Objektno orijentirane baze podataka (OODBMS) pokazale su se boljim i jeftinijim rješenjem od relacijskih baza za rad sa složenim podacima, omogućujući nasljeđivanje i integraciju s objektno orijentiranim jezicima poput C-a i Jave. Glavni nedostatak OODBMS-a bio je nedostatak standardizacije, što je otežavalo održavanje. Razvoj objektno-relacijskih baza podataka (ORDBMS) započeo je 1990-ih, kombinirajući relacijski model s konceptima OODBMS-a. ORDBMS omogućuje uvođenje novih tipova podataka i funkcija u relacijski model, koristeći SQL, uz značajke poput nasljeđivanja i polimorfizma. SQL ostaje ključni jezik za upite, što utječe na rad sustava i sučelja za programere. **foi** Za razvoj aplikacije poput e-dnevnika odabrala sam objektno-relacijsku baza podataka PostgreSQL. Ovaj izbor je motiviran sljedećim razlozima:

Kompleksnost podataka: U obrazovnom sustavu postoje brojni međusobno povezani entiteti (učenici, učitelji, predmeti, ocjene) koje je potrebno precizno modelirati. Objektno-relacijski model omogućuje prirodnije predstavljanje ovih entiteta i njihovih veza u odnosu na čisto relacijski model.

Fleksibilnost: Objektno-relacijske baze podataka omogućuju jednostavno dodavanje novih tipova podataka ili mijenjanje postojećih bez potrebe za velikim preuređenjem sheme baze. To je važno jer se obrazovni sustavi često mijenjaju, a baza podataka mora biti dovoljno fleksibilna da se prilagodi tim promjenama.

Performanse: PostgreSQL je poznat po svojim izvrsnim performansama, što je važno za aplikacije koje se intenzivno koriste.

Sigurnost: PostgreSQL nudi robustne mehanizme za kontrolu pristupa i zaštitu podataka, što je ključno za aplikacije koje obrađuju osjetljive osobne podatke.

Zrelost i podrška: PostgreSQL je zrela i dobro podržana baza podataka s velikom zajednicom korisnika, što olakšava razvoj i održavanje aplikacije.

Osim navedenih prednosti, odlučila sam se za PostgreSQL i zbog osobnog iskustva. S PostgreSQL-om radim već nekoliko godina i upoznata sam s njegovim mogućnostima, što mi omogućuje brži razvoj i održavanje aplikacije.

2.2.1. Usporedba s drugim pristupima

Relacijske baze podataka: Relacijske baze podataka su jednostavnije za upotrebu od objektno-relacijskih, ali mogu biti manje fleksibilne za modeliranje kompleksnih podataka.

NoSQL baze podataka: NoSQL baze su izvrstan izbor za velike količine neuređenih podataka, ali nisu idealne za strukturirane podatke kao što su oni u obrazovnom sustavu.

Hijerarhijske i mrežne baze podataka: Ove vrste baza su manje popularne i imaju ograničeniju primjenu u odnosu na relacijske i objektno-relacijske baze.

3. Model baze podataka

Model baze podataka za aplikaciju e-dnevnika dizajniran je kako bi se predstavile složene relacije između različitih entiteta u obrazovnom sustavu. Glavni entiteti uključuju:

Users: Predstavlja sve korisnike sustava (učitelji, učenici, roditelji, administratori). Svaki korisnik ima jedinstveni identifikator, korisničko ime, lozinku, e-mail adresu, tip korisnika (referencira tablicu UserTypes) i puno ime. Prikaz tablice u bazi vidljiv je na slici 2.

Subjects: Sadrži informacije o svim predmetima koji se izvode. Svaki predmet ima jedinstveni identifikator (subjectid) i naziv.

Grades: Čuva podatke o ocjenama koje su učenici dobili iz pojedinih predmeta. Za svaku ocjenu se bilježi student, predmet, vrijednost ocjene, datum i eventualna bilješka učitelja.

TeacherSubjects Definira koje učitelje predaju koji predmeti. Ova relacija je mnogo-prema-mnogo, što znači da jedan učitelj može predavati više predmeta, a jedan predmet može imati više učitelja.

StudentSubjects Definira koje učenike pohađaju koji predmeti. Ova relacija je također mnogo-prema-mnogo.

ParentStudents Povezuje roditelje i njihovu djecu. Ova relacija je mnogo-prema-mnogo, jer jedan roditelj može imati više djece, a jedno dijete može imati više roditelja.

UserTypes Definira različite tipove korisnika (učitelj, učenik, roditelj, administrator). Svaki korisnik pripada točno jednom tipu korisnika.

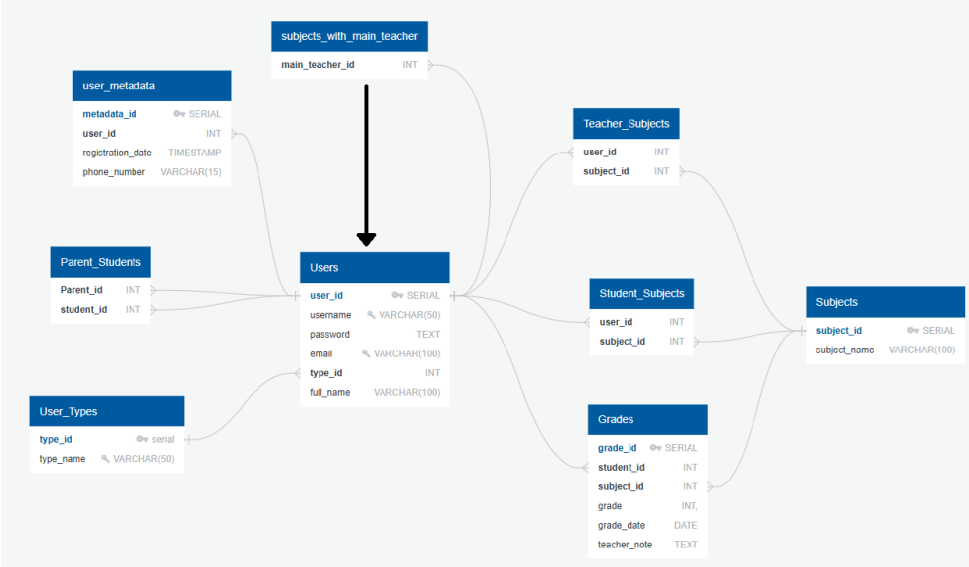
Usermetadata Sadrži dodatne metapodatke o korisnicima, kao što su datum registracije i broj telefona.

subjectswithmainteacher Ova tablica nasljeđuje atribute iz tablice Subjects i dodaje dodatni atribut MainTeacherId koji označava glavnog učitelja za taj predmet. Time se specificira koji učitelj je primarno zadužen za određeni predmet.

ERA dijagram koji prikazuje ove veze prikazan je na slici 1.

Ključne karakteristike modela:

- **Normalizacija:** Model je normaliziran kako bi se izbjegla redundancija podataka i osigurala konzistentnost.
- **Fleksibilnost:** Model omogućuje dodavanje novih tipova korisnika, predmeta i drugih entiteta bez velikih promjena u shemi baze podataka.
- **Skalabilnost:** Model može se lako proširiti kako bi se podržalo povećanje broja korisnika, predmeta i drugih podataka.



Slika 1: ERA dijagram baze podataka; vlastita izrada

user_id	username	password	email	type_id	full_name
5	kmrvi	koprvi123	mrvi@gmail.com	1	Karmelo Mrvi
6	nkrisnik	nina123	nkrsnik@gmail.com	3	Nina Krsnik
7	ivlasic	nivea123	ivalsic@gmail.com	2	Ivan Vlasic
8	lnizic	krse123	lnizic@gmail.com	3	Luka Nizic
9	vdokovic	limes123	vdokovic@gmail.com	1	Vedrana Đokovic
10	rhrkovac	rebi123	rhrkovac@gmail.com	2	Rebeka Hrkovac
11	mstura	marino123	mstura@gmail.com	2	Marino Štura
12	lkriskovic	laura123	lkriskovi@gmail.com	2	Laura Kriskovic
13	angelina	ina123	angelina@gmail.com	1	Angelina Doria
14	hegi	hegi123	hegi@gmail.com	1	Franko Hegeduš
15	salajbeg	sara123	salajbeg@gmail.com	3	Sara Alajbeg
16	rmatekovic	ruza123	rmatekovic@gmail.com	3	Ruža Matekovic
17	matucec	zeljka123	matucec@gmail.com	1	Željka Matucec
18	krebic	rebic123	krebic@gmail.com	1	Kruno Rebic
20	linmanuel	miranda	lmm@gmail.com	4	Lin Manuel Miranda

(15 rows)

Slika 2: Tablica Users; vlastita izrada

4. Implementacija

```
templates > login.html > html > head > link
<!DOCTYPE html>
<html lang="hr">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
Run and Debug (Ctrl+Shift+D)  ijava</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body>
    <div class="login-container">
        <h2>Prijavi se na svoj račun</h2>
        <form action="/login" method="POST">
            <label for="username">Korisničko ime:</label>
            <input type="text" id="username" name="username" required>

            <label for="password">Lozinka:</label>
            <input type="password" id="password" name="password" required>

            <button type="submit">Prijavi se</button>
        </form>
        {% with messages = get_flashed_messages() %}
        {% if messages %}
            <ul class="error-messages">
                {% for message in messages %}
                    <li>{{ message }}</li>
                {% endfor %}
            </ul>
        {% endif %}
    {% endwith %}
    </div>
</body>
</html>
```

Slika 3: HTML za login page; vlastita izrada

```
from flask import Flask, jsonify, render_template, request, redirect, session, flash, url_for
from flask_sqlalchemy import SQLAlchemy
from datetime import datetime

app = Flask(__name__)
app.secret_key = '1ac48208de2e01a9192cb6ae094fa1b55afc4e532829864e'
app.config['SQLALCHEMY_DATABASE_URI'] = 'postgresql://laura:laura@localhost/baza'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
db = SQLAlchemy(app)
```

Slika 4: Pocetno postavljanje i povezivanje na bazu; vlastita izrada

```
@app.route('/')
def main():
    if 'user_id' not in session:
        return redirect(url_for('login'))
    if 'user_id' in session:
        user = Users.query.get(session['user_id'])
        full_name = user.full_name

        if user.type_id == 1:
            user_role = 'professor'
            subjects = TeacherSubjects.query.filter_by(user_id=user.user_id).all()
            subject_grades = []
            students = []
            subject_students = []

            for subject in subjects:
                subject_name = Subjects.query.get(subject.subject_id).subject_name
                grades = db.session.query(Users.full_name, Grades.grade, Grades.grade_id)\
                    .join(Grades, Users.user_id == Grades.student_id)\
                    .filter(Grades.subject_id == subject.subject_id).all()

                students = db.session.query(Users.full_name, Users.user_id).join(StudentSubjects, StudentSub
                    .filter(Users.type_id == 2, StudentSubjects.subject_id == subject.subject_id).all()

                subject_grades.append({'subject_name': subject_name, 'grades': grades})

            return render_template('main.html', full_name=full_name, user_role=user_role, subject_grades=sub
```

Slika 5: Main za korisnika - učitelja; vlastita izrada

```python
        elif user.type_id == 2:
            user_role = 'student'
            student_subjects = db.session.query(Subjects.subject_name, Grades.grade).\
                join(StudentSubjects, StudentSubjects.subject_id == Subjects.subject_id).\
                join(Grades, Grades.subject_id == Subjects.subject_id).\
                filter(StudentSubjects.user_id == user.user_id).\
                all()
            subject_grades = {}
            for subject_name, grade in student_subjects:
                if subject_name not in subject_grades:
                    subject_grades[subject_name] = []
                subject_grades[subject_name].append(grade)

            subject_averages = {}
            for subject, grades in subject_grades.items():
                average = sum(grades) / len(grades)
                subject_averages[subject] = round(average, 2)

            return render_template('main.html', subject_averages=subject_averages, full_name=full_name, user_role=
```

Slika 6: Main za korisnika - učenika; vlastita izrada

```python
    elif user.type_id == 3:
        user_role = 'parent'
        # Dohvati dijete roditelja
        child = db.session.query(Users).join(ParentStudents, ParentStudents.student_id == Users.user_id)\
            .filter(ParentStudents.parent_id == user.user_id).first()
        if child:
            grades_by_subject = db.session.query(
            Subjects.subject_name,
            Grades.grade,
            Grades.teacher_note,
            Grades.subject_id
        ).join(Subjects, Subjects.subject_id == Grades.subject_id)\
        .filter(Grades.student_id == child.user_id).all()
        max_columns = 0
        subject_data = {}
        for subject_name, grade, teacher_note, subject_id in grades_by_subject:
            if subject_name not in subject_data:
                subject_data[subject_name] = {'grades': [], 'notes': [], 'average': 0}
            subject_data[subject_name]['grades'].append(grade)
            subject_data[subject_name]['notes'].append(teacher_note)
            subject_data[subject_name]['average'] = round(sum(subject_data[subject_name]['grades']) / len(subject_dat

        for subject_name, data in subject_data.items():
            grades = data['grades']
            data['average'] = round(sum(grades) / len(grades), 2) if len(grades) > 0 else None
            max_columns = max(max_columns, len(grades))


        return render_template(
            'main.html',
            full_name=full_name,
            user_role=user_role,
            child_name=child.full_name,
            subject_data=subject_data,
            max_columns=max_columns
        )
```

Slika 7: Main za korisnika - roditelja; vlastita izrada

```python
@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        username = request.form['username']
        password = request.form['password']
        user = Users.query.filter_by(username=username).first()
        if user and user.password == password:
            session['user_id'] = user.user_id
            session['type_id'] = user.type_id
            if user.type_id == 4:
                return redirect('/admin_dashboard')
            else:
                return redirect('/')
        else:
            flash('Neispravno korisničko ime ili lozinka!', 'error')
            return redirect('/login')
    return render_template('login.html')

@app.route('/logout')
def logout():
    session.clear()
    return redirect(url_for('login'))
```

Slika 8: Login i logout funkcije; vlastita izrada

```python
@app.route('/add_grade', methods=['GET', 'POST'])
def add_grade():
    if 'user_id' not in session:
        return redirect('/login')

    user = Users.query.get(session['user_id'])
    if user.type_id != 1:
        return redirect('/')

    if request.method == 'POST':
        try:
            student_id = int(request.form['student_id'])
            grade = request.form['grade']
            teacher_note = request.form.get('teacher_note', None)

            subject = db.session.query(Subjects).join(TeacherSubjects).filter(
                TeacherSubjects.user_id == user.user_id
            ).first()

            if not subject:
                return redirect('/')

            subject_id = subject.subject_id

            student = db.session.query(Users).join(StudentSubjects).filter(
                StudentSubjects.subject_id == subject_id,
                StudentSubjects.user_id == student_id
            ).first()

            if not student:
                return redirect('/')

            date = datetime.now().strftime('%Y-%m-%d')
            new_grade = Grades(
                student_id=student_id,
                subject_id=subject_id,
                grade=grade,
                teacher_note=teacher_note,
                grade_date=date
            )
```

Slika 9: Dodavanje nove ocijene; vlastita izrada

```python
@app.route('/delete_grade/<int:grade_id>', methods=['POST'])
def delete_grade(grade_id):
    if 'user_id' not in session:
        return redirect('/login')

    user = Users.query.get(session['user_id'])
    if not user or user.type_id != 1:
        flash('Pristup nije dozvoljen!', 'error')
        return redirect('/login')

    grade = Grades.query.get(grade_id)
    if not grade:
        flash('Ocjena nije pronađena!', 'error')
        return redirect('/')

    main_teacher = SubjectsWithMainTeacher.query.filter_by(subject_id=grade.subject_id, main_teach
    if not main_teacher:
        flash('Samo glavni nositelj predmeta može obrisati ocjenu!', 'error')
        return redirect('/')

    try:
        db.session.delete(grade)
        db.session.commit()
        flash('Ocjena je uspješno obrisana!', 'success')
        return jsonify({"success": True}), 200
    except Exception as e:
        db.session.rollback()
        flash('Dogodila se greška prilikom brisanja ocjene!', 'error')
        return jsonify({"success": False, "error": "Error deleting grade"}), 500
```

Slika 10: Brisanje upisane ocijene; vlastita izrada

```python
@app.route('/admin_dashboard', methods=['GET'])
def admin_dashboard():
    if 'user_id' not in session:
        return redirect('/login')

    user = Users.query.get(session['user_id'])
    if not user or user.type_id != 4:
        flash('Pristup nije dozvoljen!', 'error')
        return redirect('/')

    users_with_metadata = db.session.query(
        Users.full_name, Users.email, UserMetadata.phone_number, Users.user_id, UserMetadata.registration_date
    ).join(UserMetadata, Users.user_id == UserMetadata.user_id).all()

    return render_template('admin.html', users_with_metadata=users_with_metadata)
```

Slika 11: Prosljeđivanje podataka korisniku - adminu; vlastita izrada

```python
@app.route('/add_user', methods=['GET', 'POST'])
def add_user():
    if 'user_id' not in session or session['type_id'] != 4:
        flash('Pristup nije dozvoljen!', 'error')
        return redirect('/login')

    if request.method == 'POST':
        username = request.form.get('username')
        email = request.form.get('email')
        password = request.form.get('password')
        full_name = request.form.get('full_name')
        phone_number = request.form.get('phone_number')
        type_id = request.form.get('type_id', type=int)

        if not (username and email and password and full_name and type_id):
            flash('Sva obavezna polja moraju biti ispunjena!', 'error')
            return redirect('/add_user')
        new_user = Users(username=username, email=email, full_name=full_name, type_id=type_id, password=passwor
```

Slika 12: Dodavanje novog korisnika; vlastita izrada

```python
@app.route('/delete_user/<int:user_id>', methods=['POST'])
def delete_user(user_id):
    if 'user_id' not in session or session['type_id'] != 4:
        return jsonify({"success": False, "error": "Unauthorized"}), 403

    user = Users.query.get(user_id)
    if not user:
        return jsonify({"success": False, "error": "User not found"}), 404

    try:
        Grades.query.filter_by(student_id=user_id).delete()
        TeacherSubjects.query.filter_by(user_id=user_id).delete()
        StudentSubjects.query.filter_by(user_id=user_id).delete()
        ParentStudents.query.filter((ParentStudents.parent_id == user_id) | (ParentStudents.student_id == user_id)
        UserMetadata.query.filter_by(user_id=user_id).delete()
        SubjectsWithMainTeacher.query.filter_by(main_teacher_id=user_id).delete()

        db.session.delete(user)
        db.session.commit()
        return jsonify({"success": True}), 200
    except Exception as e:
        db.session.rollback()
        print(f"Database error: {e}")
        return jsonify({"success": False, "error": "Database error"}), 500

if __name__ == "__main__":
    with app.app_context():
        db.create_all()
    app.run(debug=True)
```

Slika 13: Brisanje postojećeg korisnika; vlastita izrada

```html
<!DOCTYPE html>
<html lang="hr">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
    <title>Dodaj Korisnika</title>
</head>
<body>
    <h1>Dodaj Novog Korisnika</h1>
    <form method="POST" action="/add_user">
        <label for="username">Korisničko ime:</label>
        <input type="text" name="username" required><br><br>

        <label for="email">E-mail:</label>
        <input type="email" name="email" required><br><br>

        <label for="password">Lozinka:</label>
        <input type="password" name="password" required><br><br>

        <label for="full_name">Ime i Prezime:</label>
        <input type="text" name="full_name" required><br><br>

        <label for="phone_number">Broj Telefona:</label>
        <input type="text" name="phone_number"><br><br>

        <label for="type_id">Tip korisnika:</label>
        <select name="type_id" required>
            <option value="1">Učitelj</option>
            <option value="2">Učenik</option>
            <option value="3">Roditelj</option>
            <option value="4">Administrator</option>
        </select><br><br>

        <button type="submit">Spremi Korisnika</button>
    </form>
</body>
</html>
```

Slika 14: HTML za stranicu dodavanje korisnika; vlastita izrada

```html
<html lang="hr">
<body>
    <div class="container">
        <table>
            <thead>
            </thead>
            <tbody>
                {% for user in users_with_metadata %}
                    <tr>
                        <td>{{ user[0] }}</td>
                        <td>{{ user[1] }}</td>
                        <td>{{ user[2] }}</td>
                        <td>{{ user[4] }}</td>
                        <td>{{ user[3] }}</td>
                        <td>
                            <button onclick="deleteUser({{ user[3] }})">Obriši</button>
                        </td>
                    </tr>
                {% endfor %}
            </tbody>
        </table>
    </div>
    <script>
        function deleteUser(userId) {
            console.log("Deleting user with ID:", userId);
            fetch(`/delete_user/${userId}`, {
                method: 'POST',
                headers: {
                    'Content-Type': 'application/json',
                },
            })
            .then(response => response.json())
            .then(data => {
                if (data.success) {
                    alert("User deleted successfully!");
                    location.reload();
                } else {
                    alert("Error deleting user: " + (data.error || "Unknown error"));
                }
            })
```

Slika 15: HTML za stranicu admina; vlastita izrada

```html
{% elif user_role == 'parent' %}
    <h2>Ocjene mog djeteta: {{ child_name }}</h2>
    <table>
        <tr>
            <th>Predmet</th>
            <th>Ocjene</th>
            <th>Bilješke</th>
            <th>Prosjek</th>
        </tr>
        {% for subject, data in subject_data.items() %}
        <tr>
            <td>{{ subject }}</td>
            <td>
                {% for grade in data.grades %}
                    {{ grade }}{% if not loop.last %}, {% endif %}
                {% endfor %}
            </td>
            <td>
                {% for note in data.notes %}
                    {{ teacher_note if teacher_note else "Nema bilješke" }}{% if not loop.last %}, {% endi
                {% endfor %}
            </td>
            <td>{{ data.average }}</td>
        </tr>
        {% endfor %}
    </table>
```

Slika 16: HTML za prikaz roditelju; vlastita izrada

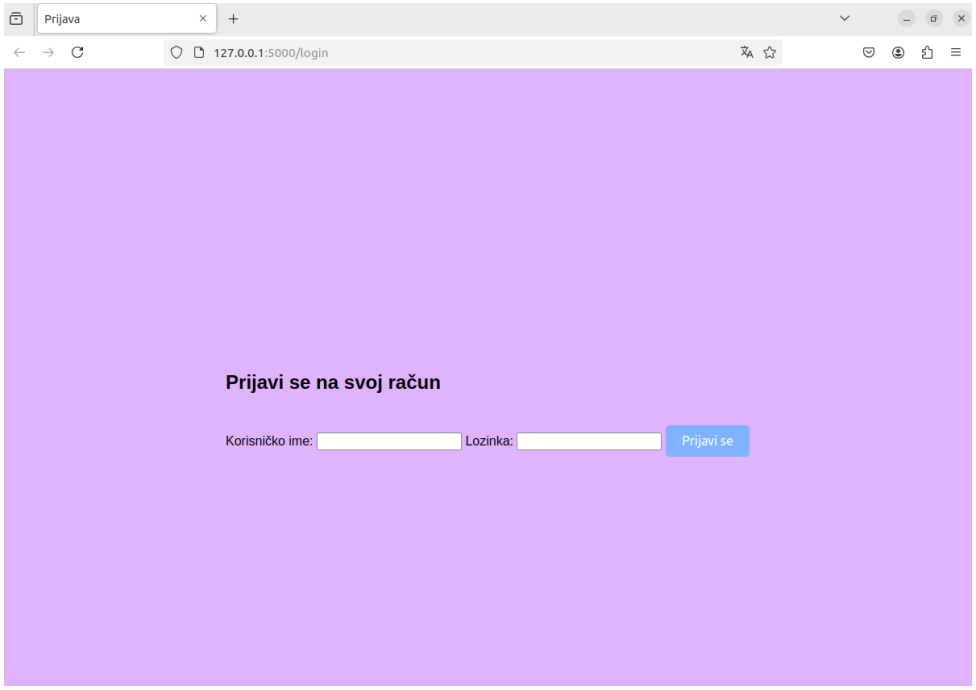
```
<div class="container">
    {% if user_role == 'professor' %}
        {% for subject in subject_grades %}
            <h2>Ocjene za predmet: {{ subject.subject_name }}</h2>
            <table>
                <tr>
                    <th>Ime i prezime</th>
                    <th>Ocjene</th>
                    <th>Delete</th>
                </tr>
                {% for grade in subject.grades %}
                <tr>
                    <td>{{ grade[0] }}</td>
                    <td>{{ grade[1] }}</td>
                    <td>
                        <button id="delete-grade" type="button" onclick="gradeDelete({{ grade[2] }})">Delete</
                    </td>
                </tr>
                {% endfor %}
            </table>
        {% endfor %}
        <button id="add-grade-button" type="button" onclick="toggleGradeForm()">Dodaj Ocjenu</button>

        <div id="grade-form" style="display: none;">
            <form action="/add_grade" method="POST">
                <label for="student_id">Odaberite učenika:</label>
                <select id="student_id" name="student_id" required style="width: 300px; height: 40px; font-siz
                    {% for student in students %}
                        <option value="{{ student[1] }}">{{ student[0] }}</option>
                    {% endfor %}
                </select>

                <label for="grade">Ocjena:</label>
                <input type="number" id="grade" name="grade" min="1" max="5" required>

                <label for="note">Bilješka:</label>
                <textarea id="note" name="note" ></textarea>
```

Slika 17: HTML za prikaz profesoru; vlastita izrada

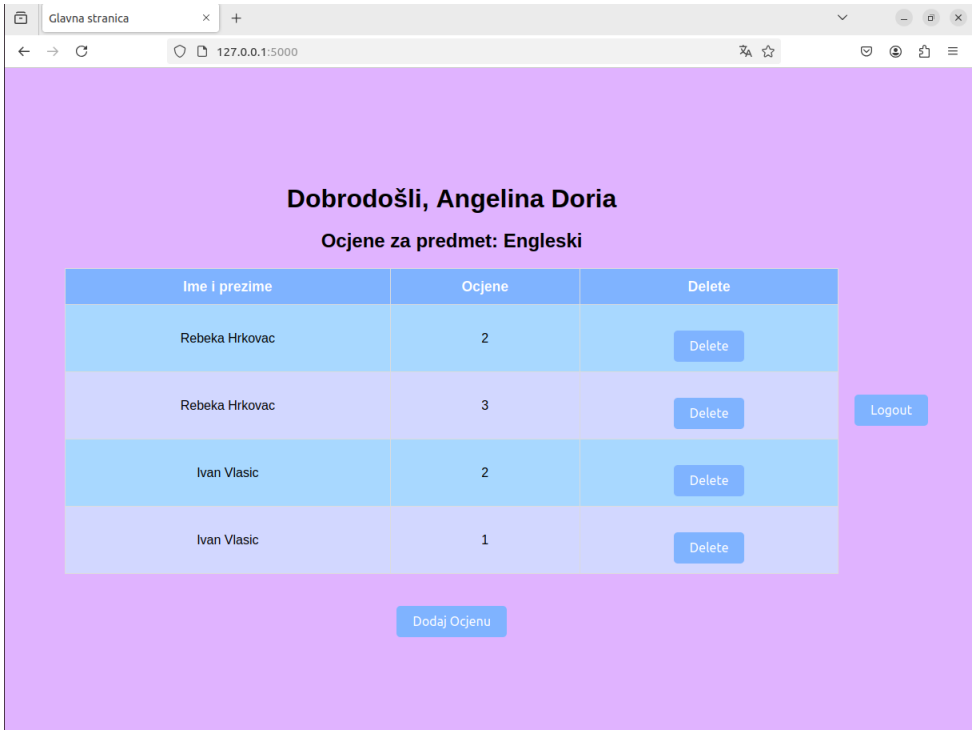
```
    </script>
{% elif user_role == 'student' %}
<h2>Moje ocjene</h2>
<table border="2px">
    <thead>
        <tr>
            <th>Predmet</th>
            <th>Ocjene</th>
            <th>Prosjek</th>
        </tr>
    </thead>
    <tbody>
        {% for subject, grades in subject_grades.items() %}
            <tr>
                <td>{{ subject }}</td>
                <td>
                    {% for grade in grades %}
                        {{ grade }}<br>
                    {% endfor %}
                </td>
                <td>{{ subject_averages[subject] }}</td>
            </tr>
        {% endfor %}
    </tbody>
</table>
```

Slika 18: HTML za prikaz učeniku; vlastita izrada

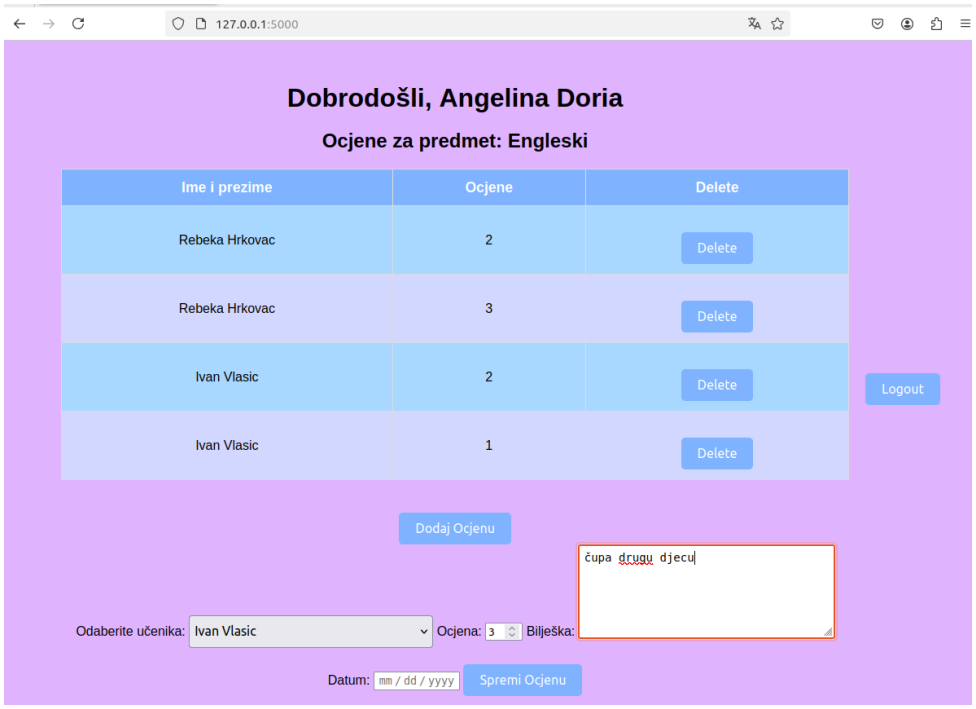
5. Primjeri korištenja



Slika 19: Prikaz prijave; vlastita izrada



Slika 20: Profesorov prikaz; vlastita izrada



Slika 21: Upis ocijene; vlastita izrada

Slika 22: Prikaz upisane ocijene; vlastita izrada

Dobrodošli, Sara Alajbeg

Ocjene mog djeteta: Rebeka Hrkovac

Predmet	Ocjene	Bilješke	Prosjek
Engleski	3, 2, 3, 2	Nema bilješke, Nema bilješke, Nema bilješke, Nema bilješke	2.5
Hrvatski	2, 5, 2, 5	Nema bilješke, Nema bilješke, Nema bilješke, Nema bilješke	3.5

Logout

Slika 23: Roditeljev prikaz; vlastita izrada

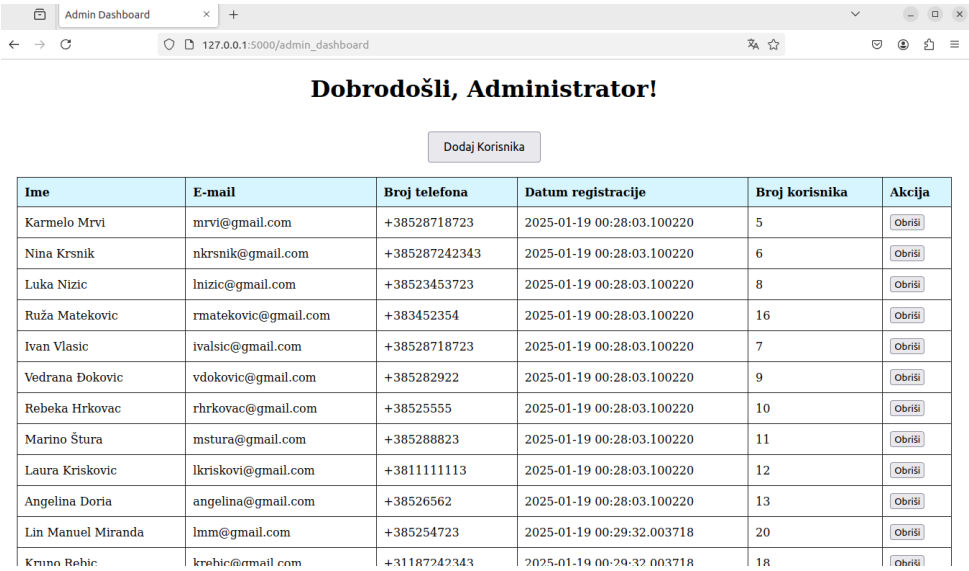
Dobrodošli, Marino Štura

Moje ocjene

Predmet	Ocjene	Prosjek
Biologija	1 1 5 5	3.0
Matematika	5 5 4 4 5 5	4.67
Povijest	4 4	4.0

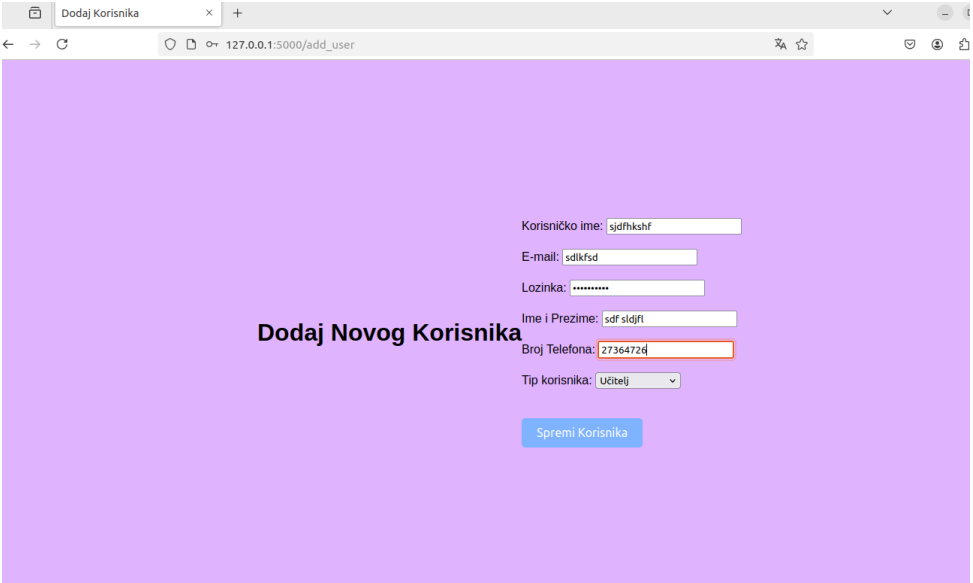
Logout

Slika 24: Ucenikov prikaz; vlastita izrada

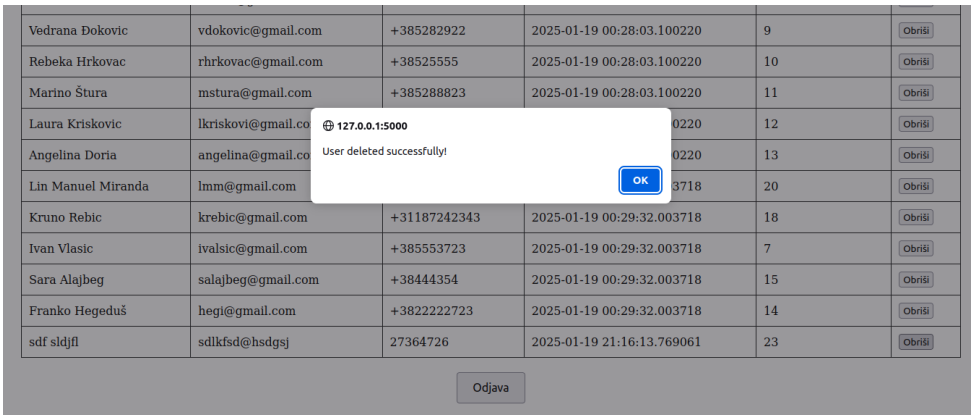


Ime	E-mail	Broj telefona	Datum registracije	Broj korisnika	Akcija
Karmelo Mrvi	mrvi@gmail.com	+38528718723	2025-01-19 00:28:03.100220	5	Obriši
Nina Krsnik	nkrsnik@gmail.com	+385287242343	2025-01-19 00:28:03.100220	6	Obriši
Luka Nizic	lnizic@gmail.com	+38523453723	2025-01-19 00:28:03.100220	8	Obriši
Ruža Matekovic	rmatekovic@gmail.com	+383452354	2025-01-19 00:28:03.100220	16	Obriši
Ivan Vlasic	ivalsic@gmail.com	+38528718723	2025-01-19 00:28:03.100220	7	Obriši
Vedrana Đokovic	vdokovic@gmail.com	+385282922	2025-01-19 00:28:03.100220	9	Obriši
Rebeka Hrkovac	rhrkovac@gmail.com	+38525555	2025-01-19 00:28:03.100220	10	Obriši
Marino Štura	mstura@gmail.com	+385288823	2025-01-19 00:28:03.100220	11	Obriši
Laura Kriskovic	lkriskovi@gmail.com	+3811111113	2025-01-19 00:28:03.100220	12	Obriši
Angelina Doria	angelina@gmail.com	+38526562	2025-01-19 00:28:03.100220	13	Obriši
Lin Manuel Miranda	lmm@gmail.com	+385254723	2025-01-19 00:29:32.003718	20	Obriši
Kruno Rebic	krebic@gmail.com	+31187242343	2025-01-19 00:29:32.003718	18	Obriši

Slika 25: Prikaz stranice admina; vlastita izrada



Slika 26: Dodavanje novog korisnika; vlastita izrada



Vedrana Đokovic	vdokovic@gmail.com	+385282922	2025-01-19 00:28:03.100220	9	Obriši
Rebeka Hrkovac	rhrkovac@gmail.com	+38525555	2025-01-19 00:28:03.100220	10	Obriši
Marino Štura	mstura@gmail.com	+385288823	2025-01-19 00:28:03.100220	11	Obriši
Laura Kriskovic	lkriskovi@gmail.co		0220	12	Obriši
Angelina Doria	angelina@gmail.co		0220	13	Obriši
Lin Manuel Miranda	lmm@gmail.com		3718	20	Obriši
Kruno Rebic	krebic@gmail.com	+31187242343	2025-01-19 00:29:32.003718	18	Obriši
Ivan Vlasic	ivalsic@gmail.com	+385553723	2025-01-19 00:29:32.003718	7	Obriši
Sara Alajbeg	salajbeg@gmail.com	+38444354	2025-01-19 00:29:32.003718	15	Obriši
Franko Hegeduš	hegi@gmail.com	+3822222723	2025-01-19 00:29:32.003718	14	Obriši
sdf sldjfl	sdlkfsd@hsdgsj	27364726	2025-01-19 21:16:13.769061	23	Obriši

Slika 27: Prikaz brisanja postojećeg korisnika; vlastita izrada

Angelina Doria	angelina@gmail.com	+38526562	2025-01-19 00:28:03.100220	13	Obriši
Lin Manuel Miranda	lmm@gmail.com	+385254723	2025-01-19 00:29:32.003718	20	Obriši
Kruno Rebic	krebic@gmail.com	+31187242343	2025-01-19 00:29:32.003718	18	Obriši
Ivan Vlasic	ivalsic@gmail.com	+385553723	2025-01-19 00:29:32.003718	7	Obriši
Sara Alajbeg	salajbeg@gmail.com	+38444354	2025-01-19 00:29:32.003718	15	Obriši
Franko Hegeduš	hegi@gmail.com	+3822222723	2025-01-19 00:29:32.003718	14	Obriši

Odjava

Slika 28: Korisnik izbrisan iz baze korisnika; vlastita izrada

6. Zaključak

Implementacija ovog sustava pokazala je kako **PostgreSQL** i **Flask** čine pouzdanu platformu za razvoj aplikacija unutar obrazovnog sustava. Objektno-relacijski model PostgreSQL-a omogućio je učinkovito modeliranje kompleksnih odnosa između korisničkih uloga, dok je Flask pružio fleksibilnost i brzinu razvoja web aplikacije.

Glavna prednost korištene tehnologije je njezina sposobnost prilagodbe specifičnim zahtjevima aplikacijske domene, poput granularne kontrole pristupa podacima i podrške za metapodatke korisnika. Također, PostgreSQL nudi visoku razinu sigurnosti, skalabilnosti i kompatibilnosti s modernim tehnologijama, dok Flask omogućava brzo proširenje funkcionalnosti i jednostavnu integraciju s drugim alatima.

Međutim, ograničenja implementacije uključuju složenost modeliranja i optimizacije baze podataka pri velikom broju korisnika te potencijalno povećano vrijeme odgovora u scenarijima s visokom učestalošću istovremenih zahtjeva. Osim toga, iako je Flask idealan za manje i srednje velike aplikacije, za veće sustave može zahtijevati dodatne alate za optimizaciju i skaliranje.

Smatram da je odabrana tehnologija pružila je čvrst temelj za implementaciju i potencijalno buduće proširenje ove aplikacije, osiguravajući ravnotežu između fleksibilnosti, sigurnosti i skalabilnosti.

Popis slika

1. ERA dijagram baze podataka; vlastita izrada 6
2. Tablica Users; vlastita izrada 6
3. HTML za login page; vlastita izrada 7
4. Pocetno postavljanje i povezivanje na bazu; vlastita izrada 7
5. Main za korisnika - učitelja; vlastita izrada 7
6. Main za korisnika - učenika; vlastita izrada 8
7. Main za korisnika - roditelja; vlastita izrada 8
8. Login i logout funkcije; vlastita izrada 8
9. Dodavanje nove ocijene; vlastita izrada 9
10. Brisanje upisane ocijene; vlastita izrada 9
11. Prosljeđivanje podataka korisniku - adminu; vlastita izrada 10
12. Dodavanje novog korisnika; vlastita izrada 10
13. Brisanje postojećeg korisnika; vlastita izrada 10
14. HTML za stranicu dodavanje korisnika; vlastita izrada 11
15. HTML za stranicu admina; vlastita izrada 12
16. HTML za prikaz roditelju; vlastita izrada 12
17. HTML za prikaz profesoru; vlastita izrada 13
18. HTML za prikaz učeniku; vlastita izrada 13
19. Prikaz prijave; vlastita izrada 14
20. Profesorov prikaz; vlastita izrada 14
21. Upis ocijene; vlastita izrada 15
22. Prikaz upisane ocijene; vlastita izrada 15
23. Roditeljev prikaz; vlastita izrada 16

24. Ucenikov prikaz; vlastita izrada 16
25. Prikaz stranice admina; vlastita izrada 17
26. Dodavanje novog korisnika; vlastita izrada 17
27. Prikaz brisanja postojećeg korisnika; vlastita izrada 17
28. Korisnik izbrisan iz baze korisnika; vlastita izrada 18

7. Prilog

Projekt se nalazi na linku: `https://github.com/lkriskovi21/projekti`